GAAO Instantiation: Personal Health & Fitness Agent

Notation Alignment with Formal Specification

This instantiation uses the cleaned GAAO formal specification with the following notation:

Core Components:

- **E** — Event Ledger Layer
- **C** — Semantic Topology Layer
- **K** — Constraint Fabric
- **X** — Condition Space Layer (X_d: dimensions, X_m: models)
- **R** — Evidential Graph Layer
- **P, Ω** — Transformation Layer
- **I** — Adaptive Reasoning Engine
- **L** — Recursive Adaptation Loop

Key Symbols:

- $\boxed{\sigma}$ — condition snapshot (formerly χ/context)
- $\boxed{\varsigma}$ — container state vector (formerly S)
- $\boxed{H\_\phi}$ — drift history (sequence of drift signals ϕ)
- $\boxed{\gamma}$ — constraint evaluation function (formerly Γ)
- $\boxed{\lambda\_c}$ — semantic container identifier (formerly c)
- $\boxed{\lambda\_m}$ — engagement mode (formerly m)
- $\boxed{\pi, \alpha}$ — planned and actual attributes (formerly A_p, A_a)
- $\boxed{\delta}$ — deviation signals
- $\boxed{\phi}$ — drift signal
- $\boxed{\tau}$ — trajectory signal

Domain Selection Rationale

We instantiate GAAO as a **personal health and fitness agent** because it:

- Involves time-extended behavior (weeks/months)

- Has clear constraints (goals, schedules, health limits)
- Requires adaptation to deviations (missed workouts, injuries)
- Benefits from condition-aware reasoning (energy levels, sleep, stress)
- Has measurable outcomes (weight, strength, consistency)
- Operates at multiple timescales (daily/weekly/monthly)

---

1. CONCRETE INSTANTIATION

1.1 Semantic Topology Layer (C)

The agent organizes life domains hierarchically:

```
Root
├── Health
│   ├── Fitness
│   │   ├── Strength_Training
│   │   ├── Cardio
│   │   └── Flexibility
│   ├── Nutrition
│   │   ├── Meal_Planning
│   │   └── Hydration
│   └── Recovery
│       ├── Sleep
│       └── Rest_Days
├── Work
│   ├── Projects
│   └── Meetings
└── Personal
    ├── Social
    └── Hobbies
```

Example Container: Strength_Training

```
c_strength = {
  id: "strength_training_001",
  τ: "activity_domain",  // container type
  parent: "fitness",

  ς: {  // container state vector
    current_program: "5x5_stronglifts",
    sessions_per_week: 3,
    current_cycle_week: 4,
    max_squat: 225,
    max_bench: 185,
    max_deadlift: 275
  },

  H_e: [/* event history */],
  H_ω: [/* outcome history */],
  H_p: [/* progress records */],
  H_ϕ: [/* drift history - sequence of ϕ signals */]
}
```

1.2 Event Ledger Layer (E)

Example Event: Monday Morning Workout

```
e_workout_mon = {
  t_s: "2025-12-08 06:00",  // ∈ T
  t_e: "2025-12-08 07:15",  // ∈ T
  λ_c: "strength_training_001",  // ∈ C (semantic container identifier)
  λ_m: "focused",  // ∈ M (engagement mode)

  π: {  // planned attributes ∈ A
    exercises: ["squat", "bench_press", "barbell_row"],
    squat_sets: 5,
    squat_weight: 225,
    squat_reps: 5,
    duration_min: 60
  },

  α: {  // actual attributes ∈ A
    exercises: ["squat", "bench_press", "barbell_row"],
    squat_sets: 5,
    squat_weight: 225,
    squat_reps: [5, 5, 5, 4, 3],  // Failed last two sets
    duration_min: 75
  },

  σ: {  // condition snapshot ∈ X (formerly χ/context)
    sleep_hours: 6.2,
    sleep_quality: 0.65,
    stress_level: 0.7,
    previous_workout: "2025-12-06",
    recovery_days: 1,
    energy_level: 0.6,
    meal_timing: "pre_workout_60min"
  },

  ω: {  // outcomes ∈ Ω
    internal_state: {
      fatigue: 0.8,
      soreness_predicted: 0.7,
      confidence: 0.6
    },
    external_state: {
      workout_completed: true,
      progression_achieved: false
    },
    state_transition: "strength_plateau",
    deviation_class: "performance_decline"
  },
```

```
  δ: {  // deviation signals ∈ D
    reps_delta: -3,  // Failed 3 total reps
    duration_delta: +15,  // Took longer
    form_quality_delta: -0.2
  },

  ϰ: ["k_strength_progression", "k_workout_consistency", "k_recovery"]  // ⊆ K
}
```

Event as formal tuple:

$$e \in E \subseteq T \times T \times C \times M \times A \times A \times X \times \Omega \times D \times \wp(K)$$

1.3 Constraint Fabric (K)

Constraint 1: Strength Progression

```
k_strength_progression = {
  id: "k_001",

  ι: {  // intention descriptor
    description: "Progressive overload in primary lifts",
    target_trajectory: "increase_by_5lb_per_week",
    min_acceptable: "maintain_current_strength",
    ideal: "linear_progression"
  },

  θ: {  // obligation specification
    metric: "total_volume_lifted",
    measurement: "sum(weight × reps × sets) per exercise per week",
    threshold_min: 15000,  // lbs
    threshold_target: 17000,
    evaluation_window: "weekly"
  },

  μ: "aggregate_weekly",  // measurement mode

  W: ["2025-01-01", "2025-06-30"],  // activation window ⊆ T

  L_c: ["strength_training_001", "recovery"],  // bound containers ⊆ C

  γ: {  // evaluation function
    fulfilled: (actual, target) => actual >= target * 0.95,
    violated: (actual, target) => actual < target * 0.85,
    trend: (history) => linear_regression(history).slope
  },

  H_k: [  // adjustment history
    {
      date: "2025-11-15",
      adjustment: "reduced_volume_by_10pct",
      reason: "persistent_fatigue_signals"
    },
    {
      date: "2025-12-01",
      adjustment: "added_deload_week",
      reason: "accumulated_fatigue"
    }
  ]
}
```

Constraint evaluation function signature:

$\gamma_k : (E, X, C) \to [0,1] \cup \{\text{fulfilled}, \text{violated}\}$

Constraint 2: Workout Consistency

```
k_workout_consistency = {
  id: "k_002",

  ι: {
    description: "Maintain 3 strength sessions per week",
    target_trajectory: "consistent_adherence"
  },

  θ: {
    metric: "sessions_per_week",
    measurement: "count(completed_workouts)",
    threshold_min: 2,
    threshold_target: 3,
    evaluation_window: "weekly"
  },

  μ: "count_weekly",
  W: ["2025-01-01", "2025-12-31"],
  L_c: ["strength_training_001"],

  γ: {
    fulfilled: (count) => count >= 3,
    violated: (count) => count < 2,
    streak: (history) => count_consecutive_successes(history)
  },

  H_k: []
}
```

Constraint 3: Recovery Balance

```
k_recovery = {
  id: "k_003",

  ι: {
    description: "Ensure adequate recovery between sessions",
    target_trajectory: "balanced_stimulus_recovery"
  },

  θ: {
    metric: "recovery_quality_score",
    measurement: "f(sleep_quality, rest_days, soreness, stress)",
    threshold_min: 0.6,
    threshold_target: 0.75,
    evaluation_window: "per_session"
  },

  μ: "continuous_per_event",
  W: ["2025-01-01", "2025-12-31"],
  L_c: ["recovery", "strength_training_001"],

  γ: {
    fulfilled: (score) => score >= 0.75,
    warning: (score) => score >= 0.6 && score < 0.75,
    violated: (score) => score < 0.6
  },

  H_k: []
}
```

1.4 Condition Space Layer (X = X_d ∪ X_m)

Condition Dimensions (X_d):

```
ξ_sleep = {
  name: "sleep_hours_last_night",
  type: "physiological",
  value: 6.2,
  t: "2025-12-08 06:00",
  source: "sleep_tracker",
  conf: 0.9,  // confidence
  exp: "24h"  // expiry
}

ξ_stress = {
  name: "perceived_stress",
  type: "psychological",
  value: 0.7,  // 0-1 scale
  t: "2025-12-08 06:00",
  source: "self_report",
  conf: 0.7,
  exp: "12h"
}

ξ_recovery = {
  name: "recovery_status",
  type: "composite",
  value: 0.65,
  t: "2025-12-08 06:00",
  source: "recovery_pack",
  conf: 0.8,
  exp: "6h"
}
```

Condition Model (X_m): Recovery Assessment

```
M_recovery = {
  name: "recovery_quality_pack",
  type: "composite_calculator",

  inputs: [  // ⊆ X_d
    "sleep_hours",
    "sleep_quality",
    "days_since_last_workout",
    "soreness_level",
    "stress_level",
    "hrv_score"
  ],

  rules: {  // interpretive logic
    base_score: (sleep_hrs) => min(1.0, sleep_hrs / 8),
    sleep_weight: 0.35,
    rest_weight: 0.25,
    stress_weight: 0.20,
    soreness_weight: 0.20
  },

  update: "on_demand",  // evolution function
  confidence: "weighted_avg_of_inputs",  // model-level confidence
  version: "v2.1"
}
```

Condition profile at time t:

$$X_t \subseteq X$$

Update function:

$$update_M : (X, R) \rightarrow X$$

1.5 Evidential Graph Layer (R)

Evidence Record:

$$r = (id, type, t, c, k, e, raw, derived, conf, src)$$

Delta Record:

```
Δ_reps = {
  field: "squat_reps",
  planned: [5, 5, 5, 5, 5],
  actual: [5, 5, 5, 4, 3],
  delta: -3,  // δ = f(π, α) ∈ D
  delta_pct: -0.12,
  significance: "moderate"
}
```

Drift Signal (ϕ):

```
ϕ_strength_plateau = {
  type: "performance_stagnation",
  magnitude: 0.3,
  recurrence: 4,  // 4 consecutive suboptimal sessions
  link: ["strength_training_001"],  // linked containers
  t_1: "2025-11-25",  // first occurrence
  t_n: "2025-12-08"   // last occurrence
}
```

Trajectory Signal (τ):

```
τ_fatigue_accumulation = {
  pattern: "increasing_fatigue_markers",
  container: "strength_training_001",
  direction: "negative",
  confidence: 0.85,
  window: ["2025-11-15", "2025-12-08"],
  indicators: [
    "declining_performance",
    "increased_perceived_exertion",
    "reduced_sleep_quality",
    "elevated_stress"
  ]
}
```

Formal operators:

```
f_δ : (π, α) → D  // delta extraction
Drift : R → {ϕ₁, ..., ϕ_n}  // drift extraction
𝓣_r : R → {τ₁, ..., τ_m}  // trajectory extraction
```

1.6 Transformation Layer (P, Ω)

Progress Record (P):

Let Λ be the metric space.

```
p_volume = {
  c: "strength_training_001",  // ∈ C
  metric: "weekly_volume",  // ∈ Λ
  v: 16200,  // magnitude (lbs)
  d: -1,  // direction ∈ {-1, 0, 1}
  e: "e_workout_mon",  // originating event ∈ E
  t: "2025-12-08 07:15",  // timestamp ∈ T

  context: {
    vs_last_week: -800,
    vs_target: -800,
    trend_4wk: "declining"
  }
}

p_consistency = {
  c: "strength_training_001",
  metric: "adherence_rate",
  v: 1.0,  // 3/3 this week
  d: 0,  // stable
  e: "e_workout_mon",
  t: "2025-12-08 07:15"
}
```

Outcome Record (Ω):

$$\omega = (i, x, s, \delta)$$

Moment-level outcome (single workout):

```
ω_moment = {
  i: {  // internal effect
    fatigue_level: 0.8,
    confidence_score: 0.6,
    motivation_impact: -0.1
  },
  x: {  // external effect
    workout_logged: true,
    volume_recorded: 16200
  },
  s: "strength_plateau_entered",  // state-transition marker
  δ: "performance_decline"  // deviation classification ∈ D
}
```

Constraint-level outcome (weekly evaluation):

```
ω_constraint = {
  constraint: "k_strength_progression",
  period: "week_49_2025",
  fulfillment_status: "warning",
  actual_vs_target: 0.92,  // 92% of target
  trend: "declining",
  recommendation: "intervention_needed"
}
```

Trajectory-level outcome (monthly assessment):

```
ω_trajectory = {
  container: "strength_training_001",
  period: "november_2025",
  overall_direction: "plateau_with_fatigue",
  key_patterns: [
    "accumulated_fatigue",
    "performance_decline",
    "maintained_consistency"
  ],
  strategic_adjustment_needed: true
}
```

---

2. ADAPTIVE REASONING ENGINE (I)

Operator signature:

$$I : (X, E, K, C, R) \rightarrow \{\Pi, \Delta, S, Y\}$$

Where:

- **Π** — plan proposals
- **Δ** — adjustments
- **S** — simulations
- **Y** — interpretive summaries

2.1 Interpretation Function

$I_{int}: R \times X \rightarrow Y$

python

```python
def interpret_workout_session(evidence, condition_space):
    """
    Interprets a workout session by analyzing deltas,
    condition factors, and trajectory signals

    Args:
        evidence: R (evidential graph records)
        condition_space: X (condition profile)

    Returns:
        Y (interpretive summary)
    """

    # Extract key evidence
    reps_delta = evidence.deltas['squat_reps']  # δ ∈ D
    duration_delta = evidence.deltas['duration']

    # Extract condition dimensions
    sleep_quality = condition_space['sleep_quality']  # ξ ∈ X_d
    stress_level = condition_space['stress_level']
    recovery_score = condition_space['recovery_status']

    # Pattern matching
    performance_gap = abs(reps_delta) / planned_reps

    interpretation = {
        'primary_issue': None,
        'contributing_factors': [],
        'severity': None,
        'confidence': 0.0
    }

    # Decision tree interpretation
    if performance_gap > 0.15:  # >15% performance decline
        if recovery_score < 0.65:
            interpretation['primary_issue'] = 'insufficient_recovery'
            interpretation['severity'] = 'moderate'
            interpretation['confidence'] = 0.85

            if sleep_quality < 0.7:
                interpretation['contributing_factors'].append('poor_sleep')
            if stress_level > 0.65:
                interpretation['contributing_factors'].append('elevated_stress')

        elif check_trajectory_signal('fatigue_accumulation'):  # τ ∈ Φ
            interpretation['primary_issue'] = 'accumulated_fatigue'
```

```
            interpretation['severity'] = 'moderate_to_high'
            interpretation['confidence'] = 0.80

        else:
            interpretation['primary_issue'] = 'natural_plateau'
            interpretation['severity'] = 'low'
            interpretation['confidence'] = 0.60

    return interpretation  # ∈ Y
```

2.2 Pattern Detection

I_pat: R → Φ

Where Φ denotes the pattern space (drift and trajectory patterns).

python

```
def detect_drift_patterns(evidence_records, window_days=21):
    """
    Detects drift patterns in recent event history

    Args:
        evidence_records: R
        window_days: temporal window

    Returns:
        Φ (pattern space: drift ϕ and trajectory τ signals)
    """

    recent_events = filter_by_timewindow(evidence_records, window_days)

    patterns = []  # Will contain ϕ and τ signals

    # Performance trend analysis
    completion_rates = [
        e.α['reps'] / e.π['reps']  # actual / planned
        for e in recent_events
    ]

    trend = calculate_trend(completion_rates)

    if trend['slope'] < -0.05:  # Declining >5% per week
        patterns.append({
            'type': 'performance_decline',  # ϕ type
            'magnitude': abs(trend['slope']),
            'confidence': trend['r_squared'],
            'duration_days': window_days
        })

    # Recovery pattern analysis
    recovery_scores = [
        calculate_recovery_score(e.σ)  # condition snapshot
        for e in recent_events
    ]

    if mean(recovery_scores) < 0.65:
        patterns.append({
            'type': 'chronic_underrecovery',  # ϕ type
            'magnitude': 0.65 - mean(recovery_scores),
            'confidence': 0.75,
            'duration_days': window_days
        })
```

```
    # Consistency analysis
    scheduled_sessions = expected_sessions(window_days, frequency=3)
    actual_sessions = len(recent_events)
    adherence_rate = actual_sessions / scheduled_sessions

    if adherence_rate >= 0.9:
        patterns.append({
            'type': 'high_consistency',  # τ type
            'magnitude': adherence_rate,
            'confidence': 1.0
        })

    return patterns  # ∈ Φ
```

2.3 Planning Function

I_plan: K × X × C → Π

python

```
def generate_weekly_plan(constraints, condition_space, containers, current_date):
    """
    Generates next week's training plan based on
    constraints, current condition space, and container states

    Args:
        constraints: K (constraint fabric)
        condition_space: X (condition profile)
        containers: C (semantic topology)
        current_date: t ∈ T

    Returns:
        Π (plan proposal)
    """

    plan = {
        'week_starting': current_date,
        'sessions': [],
        'adjustments': [],
        'rationale': []
    }

    # Get current state
    strength_state = containers['strength_training_001'].ς  # container state vector
    recovery_score = condition_space['recovery_status']  # ξ ∈ X_d

    # Check constraint status using γ evaluation function
    progression_status = constraints['k_strength_progression'].γ(E, X, C)
    consistency_status = constraints['k_workout_consistency'].γ(E, X, C)

    # Check for drift flags
    active_drifts = get_active_drift_flags(containers['strength_training_001'].H_ϕ)

    # Decision logic
    if 'accumulated_fatigue' in [ϕ.type for ϕ in active_drifts]:
        # DELOAD WEEK
        plan['sessions'] = [
            {
                'day': 'monday',
                'type': 'strength_reduced',
                'volume_pct': 0.6,  # 60% of normal
                'intensity_pct': 0.7,
                'duration_min': 45
            },
            {
                'day': 'wednesday',
```

```
            'type': 'active_recovery',
            'activity': 'light_cardio',
            'duration_min': 30
        },
        {
            'day': 'friday',
            'type': 'strength_reduced',
            'volume_pct': 0.6,
            'intensity_pct': 0.7,
            'duration_min': 45
        }
    ]

    plan['adjustments'].append({
        'type': 'deload_week',
        'reason': 'accumulated_fatigue_detected',
        'duration': '1_week',
        'expected_outcome': 'recovery_and_supercompensation'
    })

    plan['rationale'].append(
        "Implementing deload week due to 4-week fatigue accumulation pattern. "
        "Reducing volume/intensity to promote recovery while maintaining movement patterns."
    )

elif progression_status == 'plateau' and recovery_score > 0.75:
    # PROGRESSION PUSH
    plan['sessions'] = generate_progressive_sessions(
        current_weights=strength_state['current_weights'],
        increment=5,  # lbs
        volume='normal'
    )

    plan['adjustments'].append({
        'type': 'weight_increase',
        'reason': 'plateau_with_good_recovery',
        'increment_lbs': 5
    })

else:
    # MAINTAIN CURRENT PROGRAM
    plan['sessions'] = generate_standard_sessions(
        current_weights=strength_state['current_weights'],
        volume='normal'
    )
```

```
    return plan  # ∈ Π
```

2.4 Adjustment Function

$I_{adj}: K \times C \times X \times R \to \Delta$

```python
```

```python
def adjust_system_state(constraints, containers, condition_space, evidence):
    """
    Adjusts constraints, containers, and condition space based on evidence

    Args:
        constraints: K
        containers: C
        condition_space: X
        evidence: R

    Returns:
        Δ (adjustments)
    """

    # Interpret evidence first
    interpretation = I_int(evidence, condition_space)  # → Y

    adjustments = {
        'constraints': [],
        'containers': [],
        'condition_models': []
    }

    if interpretation['primary_issue'] == 'accumulated_fatigue':

        # Adjust strength progression constraint
        k_prog = constraints['k_strength_progression']
        k_prog.θ['threshold_target'] *= 0.9  # Reduce target by 10%
        k_prog.H_k.append({
            'date': current_date(),
            'adjustment': 'reduced_volume_target_10pct',
            'reason': 'accumulated_fatigue_intervention',
            'temporary': True,
            'duration_weeks': 2
        })
        adjustments['constraints'].append(k_prog)

        # Update container state
        c_strength = containers['strength_training_001']
        c_strength.ς['program_phase'] = 'deload'  # Update state vector
        c_strength.ς['deload_weeks_remaining'] = 1

        # Add to drift history H_ϕ
        c_strength.H_ϕ.append({
            'type': 'accumulated_fatigue',
            'detected': current_date(),
```

```
            'intervention': 'deload_prescribed',
            'expected_resolution': current_date() + timedelta(weeks=2)
        })
        adjustments['containers'].append(c_strength)

        # Adjust condition model
        M_recovery = condition_space.models['recovery_quality_pack']  # ∈ X_m
        M_recovery.rules['stress_weight'] = 0.25  # Increase stress consideration
        adjustments['condition_models'].append(M_recovery)

    return adjustments  # ∈ Δ
```

2.5 Simulation Function

I_sim: state × window → S

python

```
def simulate_plan_outcomes(current_state, proposed_plan, horizon_weeks=4):
    """
    Simulates potential outcomes of proposed plan

    Args:
        current_state: (K, C, X, E, R)
        proposed_plan: Π
        horizon_weeks: simulation window

    Returns:
        S (simulation results)
    """

    simulation = {
        'scenarios': [],
        'expected_value': None,
        'risk_assessment': {}
    }

    # Scenario 1: Optimistic (good recovery, consistent execution)
    scenario_optimistic = {
        'probability': 0.25,
        'assumptions': {
            'adherence': 1.0,
            'recovery': 0.85,
            'stress': 0.4
        },
        'projected_outcomes': {
            'volume_increase': 1.15,
            'strength_gain': 1.10,
            'constraint_fulfillment': 1.0
        }
    }

    # Scenario 2: Expected (normal conditions)
    scenario_expected = {
        'probability': 0.50,
        'assumptions': {
            'adherence': 0.9,
            'recovery': 0.70,
            'stress': 0.6
        },
        'projected_outcomes': {
            'volume_increase': 1.05,
            'strength_gain': 1.03,
            'constraint_fulfillment': 0.90
```

```
    }
}

# Scenario 3: Pessimistic (poor recovery, life stress)
scenario_pessimistic = {
    'probability': 0.25,
    'assumptions': {
        'adherence': 0.7,
        'recovery': 0.55,
        'stress': 0.8
    },
    'projected_outcomes': {
        'volume_increase': 0.95,
        'strength_gain': 0.98,
        'constraint_fulfillment': 0.70,
        'risk_of_overtraining': 0.3
    }
}

simulation['scenarios'] = [
    scenario_optimistic,
    scenario_expected,
    scenario_pessimistic
]

# Calculate expected value
simulation['expected_value'] = sum(
    s['probability'] * s['projected_outcomes']['strength_gain']
    for s in simulation['scenarios']
)

# Risk assessment
simulation['risk_assessment'] = {
    'overtraining_risk': 0.25 * 0.3,  # Probability × Impact
    'injury_risk': estimate_injury_risk(current_state, proposed_plan),
    'plateau_risk': 0.15
}

return simulation  # ∈ S
```

---

3. RECURSIVE ADAPTATION LOOP (L)

Loop schema:

$$L = \{\text{Plan} \rightarrow \text{Execute} \rightarrow \text{Log} \rightarrow \text{Interpret} \rightarrow \text{Adjust}\}$$

Operational state at time t:

$$\text{state}_t = (K_t, C_t, X_t, E_t, R_t)$$

State-transition operator:

$$\text{state}_{t+1} = \mathcal{L}(\text{state}_t)$$

Complete Cycle Walkthrough

Cycle Period: Week of December 8-14, 2025

Phase 1: PLAN (Monday morning)

Input State (t=0):
state_t = (K_t, C_t, X_t, E_t, R_t)

Where:
- K_t: k_strength_progression status = "warning" (92% of target last week)
- C_t: c_strength_training.H_ϕ contains drift flag (fatigue_accumulation, 4 events)
- X_t: recovery_score = 0.65 (suboptimal)
- R_t: Contains trajectory signal τ_fatigue_accumulation (confidence 0.85)

Planning Process:
→ I_plan(K_t, X_t, C_t) executed // I_plan: K × X × C → Π

→ Detects: accumulated_fatigue pattern from H_ϕ
→ Decision: Implement deload week

Output Plan (P_t ∈ Π):
```
{
  week_type: "deload",
  sessions: [
    Monday: Strength 60% volume, 70% intensity
    Wednesday: Active recovery
    Friday: Strength 60% volume, 70% intensity
  ],
  adjustments: [
    "reduce_volume_40pct",
    "reduce_intensity_30pct",
    "prioritize_recovery"
  ],
  expected_outcomes: {
    recovery_improvement: 0.80,
    readiness_for_progression: "next_week"
  }
}
```

Phase 2: EXECUTE (Throughout week)

Events generated:

Monday Session:

```
e_mon = {
  t_s: "2025-12-08 06:00",
  t_e: "2025-12-08 06:45",
  λ_c: "strength_training_001",
  λ_m: "moderate_effort",
  π: { squat: 135lbs × 5 × 5, bench: 130lbs × 5 × 5 },
  α: { squat: 135lbs × 5 × 5, bench: 130lbs × 5 × 5 },
  σ: { sleep: 7.1, stress: 0.6, recovery: 0.68 },
  ω: { felt_easier_than_expected: true, good_energy: true },
  δ: { rpe_delta: -1.5 },  // easier than expected
  ϰ: ["k_strength_progression", "k_recovery"]
}
```

Wednesday Session:

```
e_wed = {
  t_s: "2025-12-10 17:00",
  t_e: "2025-12-10 17:30",
  λ_c: "recovery",
  λ_m: "low_intensity",
  π: { activity: "30min_moderate_cardio" },
  α: { activity: "30min_moderate_cardio", actual_duration: 30 },
  σ: { sleep: 7.8, stress: 0.5, recovery: 0.72 },
  ω: { perceived_exertion: 3/10 },
  δ: {},
  ϰ: ["k_recovery"]
}
```

Friday Session:

```
e_fri = {
  t_s: "2025-12-12 06:00",
  t_e: "2025-12-12 06:45",
  λ_c: "strength_training_001",
  λ_m: "moderate_effort",
  π: { squat: 135lbs × 5 × 5, bench: 130lbs × 5 × 5 },
  α: { squat: 135lbs × 5 × 5, bench: 130lbs × 5 × 5 },
  σ: { sleep: 7.5, stress: 0.5, recovery: 0.78 },
  ω: { strong_session: true, felt_recovered: true },
  δ: { rpe_delta: -2.0 },  // significantly easier
  ϰ: ["k_strength_progression", "k_recovery"]
}
```

Event Ledger Updated:

$$E_{t+1} = E_t \cup \{e_{mon}, e_{wed}, e_{fri}\}$$

Phase 3: LOG (Real-time + End of week)

Evidence Records Created (R):

```
r_1 = {
  id: "evidence_mon_001",
  type: "DeltaEvidence",
  t: "2025-12-08 06:45",
  c: "strength_training_001",
  k: ["k_strength_progression"],
  e: "e_mon",
  raw: { completion_rate: 1.0 },
  derived: {
    rpe_delta: -1.5,  // δ = f(π, α)
    recovery_delta: +0.03
  },
  conf: 0.85,
  src: "workout_log"
}

r_2 = {
  id: "evidence_fri_001",
  type: "DeltaEvidence",
  t: "2025-12-12 06:45",
  c: "strength_training_001",
  k: ["k_strength_progression"],
  e: "e_fri",
  raw: { completion_rate: 1.0 },
  derived: {
    rpe_delta: -2.0,  // δ = f(π, α)
    recovery_delta: +0.13
  },
  conf: 0.85,
  src: "workout_log"
}

r_3 = {
  id: "progress_week_001",
  type: "ProgressEvidence",
  t: "2025-12-14",
  c: "strength_training_001",
  k: null,
  e: null,
  raw: null,
  derived: {
    recovery_score_avg: 0.73,
    direction: +1,  // improving
    vs_previous_week: +0.13
```

```
  },
  conf: 0.80,
  src: "weekly_aggregation"
}

r_4 = {
  id: "progress_adherence_001",
  type: "ProgressEvidence",
  t: "2025-12-14",
  c: "strength_training_001",
  k: ["k_workout_consistency"],
  e: null,
  raw: null,
  derived: {
    adherence_rate: 1.0,
    direction: 0  // maintained
  },
  conf: 1.0,
  src: "weekly_count"
}

r_5 = {
  id: "trajectory_recovery_001",
  type: "TrajectorySignal",
  t: "2025-12-14",
  c: "strength_training_001",
  k: null,
  e: null,
  raw: null,
  derived: {
    pattern: "recovery_phase_successful",  // τ ∈ Φ
    fatigue_markers: "declining",
    readiness: "improved"
  },
  conf: 0.85,
  src: "pattern_detection"
}

Updated Evidential Graph:
R_{t+1} = R_t ∪ {r_1, r_2, r_3, r_4, r_5}
```

Phase 4: INTERPRET (End of week)

Interpretation Process:
→ I_int(R_{t+1}, X_{t+1}) executed // I_int: R × X → Y

Analysis using evidential operators:
- Drift_extraction: Drift(R_{t+1}) → {ϕ_fatigue_resolved}
- Trajectory_extraction: $\mathcal{T}$_r(R_{t+1}) → {τ_recovery_successful}
- All sessions completed as planned ✓
- RPE significantly lower than recent weeks
- Recovery scores trending upward
- No new negative deltas δ
- Fatigue markers declining

Interpretation Output (S_{week} ∈ Y):
```
{
  status: "deload_successful",
  confidence: 0.90,
  key_findings: [
    "Recovery objective achieved",
    "Fatigue accumulation resolved",
    "Readiness for progression restored"
  ],
  risk_factors: [],
  recommendation: "resume_progressive_loading_next_week"
}
```

Phase 5: ADJUST (End of week)

Adjustment Process:
→ I_adj(K_t, C_t, X_t, R_{t+1}) executed // I_adj: K × C × X × R → Δ

Adjustments Made (Δ):

1. Constraint Update:
k_strength_progression.θ['threshold_target'] = 17000 → 17500 // increase target
k_strength_progression.H_k.append({
  date: "2025-12-14",
  adjustment: "target_increased_post_deload",
  reason: "successful_recovery_intervention",
  expected: "resumed_progression"
})

2. Container Update:
c_strength = containers['strength_training_001']
c_strength.ς['program_phase'] = "deload" → "progression" // state vector
c_strength.ς['next_increment'] = 5 // lbs
c_strength.H_ϕ[-1]['resolved'] = "2025-12-14" // Mark drift resolved

3. Condition Model Update:
M_recovery = condition_space.models['recovery_pack'] // ∈ X_m
M_recovery.baseline_updated = true
M_recovery.reference_score = 0.73
// Update function: update_M: (X, R) → X

4. New Plan Generated (Π_{t+1}):
{
  week_type: "progressive_loading",
  sessions: [
    Monday: Squat 230lbs (↑5), Bench 190lbs (↑5)
    Wednesday: Deadlift 280lbs (↑5)
    Friday: Squat 230lbs, Bench 190lbs
  ],
  rationale: "Resume progression after successful deload"
}

State Transition:
state_{t+1} = $\mathcal{L}$(state_t)

Where:
state_{t+1} = (K_{t+1}, C_{t+1}, X_{t+1}, E_{t+1}, R_{t+1})

With:
- K_{t+1}: Constraint thresholds updated
- C_{t+1}: Container states and drift history updated

- X_{t+1}: Condition models recalibrated
- E_{t+1}: Event ledger extended
- R_{t+1}: Evidential graph extended with new records

Loop Continues...

Next Cycle (Week Dec 15-21):
state_{t+1} fed back into $\mathcal{L}$

→ PLAN: Execute I_plan(K_{t+1}, X_{t+1}, C_{t+1}) → Π_{t+2}
→ EXECUTE: Generate new events → E_{t+2}
→ LOG: Capture evidence → R_{t+2}
→ INTERPRET: I_int(R_{t+2}, X_{t+2}) → Y_{t+2}
→ ADJUST: I_adj(...) → Δ_{t+2}

Recursive adaptation continues indefinitely.

---

4. TIMESCALE ANALYSIS

The loop L operates at three scales:

Micro-scale (Single Event)

- **Duration**: 1-2 hours
- **Plan**: Specific workout
- **Execute**: Perform session
- **Log**: Record deltas δ
- **Interpret**: "How did this session go?"
- **Adjust**: Modify next session if needed
- **State transition**: e → r → p/ω

Cycle-scale (Week)

- **Duration**: 7 days
- **Plan**: Weekly training block Π
- **Execute**: Complete all planned sessions
- **Log**: Aggregate weekly progress

- **Interpret**: "Is my program working?" (Y)
- **Adjust**: Modify next week's plan (Δ)
- **Evidential operators**: Drift(R), $\mathcal{T}$_r(R)

Macro-scale (Month/Quarter)

- **Duration**: 30-90 days
- **Plan**: Training phase (hypertrophy, strength, etc.)
- **Execute**: Complete full mesocycle
- **Log**: Trajectory analysis τ
- **Interpret**: "Am I progressing toward my goal?"
- **Adjust**: Change program structure
- **Constraint evaluation**: γ_k over extended windows

---

5. COMPUTATIONAL COMPLEXITY

Storage Requirements

Per Event e ∈ E: ~2KB
Per Week: ~6KB (3 events)
Per Year: ~300KB

Containers C: ~10KB total (hierarchical structure)
Condition Space X: ~5KB active state (X_t ⊆ X)
Constraints K: ~3KB per constraint

Annual Storage: ~500KB for complete agent state

Computational Cost per Cycle

Planning Phase I_plan:

- Constraint evaluation γ: O(|K| × |E_recent|)
- Pattern detection I_pat: O(|E_window|)
- Decision tree traversal: O(log n)
- **Total**: ~50-200ms for typical agent

Interpretation Phase I_int:

- Delta calculation f_δ: O(|fields|)

- Condition aggregation: O(|X_d|)
- Pattern matching: O(|Φ|)
- **Total**: ~20-100ms

Adjustment Phase I_adj:

- Constraint updates: O(|K_affected|)
- Container state updates ς: O(|C_affected|)
- Condition model updates: O(|X_m affected|)
- **Total**: ~10-50ms

Full Cycle Cost: 100-400ms (negligible for human-timescale agent)

Scalability

- **Single Agent**: Easily handles 1000s of events, 100s of constraints
- **Multi-Agent**: Each agent independent, O(n) scaling
- **Long-term Memory**: Efficient with event pruning (keep recent detailed, aggregate historical)
- **Evidential Graph R**: Grows linearly with events but can be compacted

---

6. FORMAL PROPERTIES DEMONSTRATED

This instantiation demonstrates key GAAO properties:

6.1 Event Sourcing

Property: State is reconstructible from event ledger E

$$\text{state}_t = \text{reconstruct}(E, t)$$
$$\forall\, t \in T: \text{state}_t \text{ uniquely determined by } \{e \in E : e.t_e \leq t\}$$

Demonstrated: Container state ς can be rebuilt from H_e

6.2 Semantic Preservation

Property: Container hierarchy G preserves semantic relationships

$$G = (C, \text{parent}) \text{ is a rooted tree}$$
$$\forall\, c \in C: c.\text{parent} \in C \cup \{\varnothing\}$$

Demonstrated: Fitness → Strength_Training → (events, outcomes, progress)

6.3 Constraint Consistency

Property: Constraint evaluation is well-defined

$$\forall k \in K: \gamma_k: (E, X, C) \rightarrow [0,1] \cup \{\text{fulfilled}, \text{violated}\}$$

Demonstrated: γ functions map evidence and condition to fulfillment status

6.4 Evidential Completeness

Property: All state transitions leave evidence traces

$$\forall \text{ state transition}: \exists r \in R \text{ recording the transition}$$

Demonstrated: Every adjustment appends to H_k, H_ϕ, or updates R

6.5 Adaptive Convergence

Property: Recursive loop $\mathcal{L}$ improves constraint alignment over time

$$\lim_{n\rightarrow\infty} \text{distance}(\text{state}_n, \text{constraint\_fulfillment}) \rightarrow 0$$
(under appropriate conditions)

Demonstrated: Deload cycle resolved drift ϕ, restored progression capacity

---

7. MAPPING TO FORMAL SPECIFICATION

Complete formal mapping:

Instantiation Element	Formal Symbol	Set/Space
Event	e	$E \subseteq T \times T \times C \times M \times A \times A \times X \times \Omega \times D \times \wp(K)$
Container	c	C with topology G = (C, parent)
Container state	current_program, max_squat, etc.	ς (state vector)
Constraint	k_strength_progression	K with γ evaluation
Planned attributes	squat: 225lbs × 5 × 5	$\pi \in A$
Actual attributes	squat: [5,5,5,4,3]	$\alpha \in A$
Condition snapshot	sleep: 6.2, stress: 0.7	$\sigma \in X$ (condition profile X_t)
Condition dimension	sleep_hours, recovery_status	$\xi \in X_d$
Condition model	recovery_quality_pack	$M \in X_m$
Delta	reps_delta: -3	$\delta = f_\delta(\pi, \alpha) \in D$
Drift signal	fatigue_accumulation (4 events)	ϕ with (type, magnitude, recurrence, link, t_1, t_n)
Trajectory signal	recovery_phase_successful	$\tau \in \Phi$
Progress record	weekly_volume: 16200	p = (c, metric, v, d, e, t)
Outcome	workout_completed, plateau_entered	ω = (i, x, s, δ)
Evidence record	interview, delta, progress	$r \in R$
Plan	deload_week, progressive_loading	Π (from I_plan)
Interpretation	"deload_successful"	Y (from I_int)
Adjustment	reduce_target_10pct	Δ (from I_adj)
State at time t	All K, C, X, E, R at time t	state_t = (K_t, C_t, X_t, E_t, R_t)
Loop operator	Plan → Execute → Log → Interpret → Adjust	$\mathcal{L}$: state_t → state_{t+1}

---

8. NEXT STEPS

This instantiation provides:

✅ Complete formal alignment with cleaned GAAO specification
✅ Concrete examples of all components (E, C, K, X, R, P, Ω, I, L)
✅ Runtime semantics for I operators
✅ Full cycle walkthrough showing state_t → state_{t+1}
✅ Computational complexity analysis
✅ Formal properties demonstrated

Ready for publication as Section 4.1: Personal Health & Fitness Agent

Shall I now update the Startup Founder instantiation with the same formal alignment?